

UNIVERSIDAD NACIONAL DE CÓRDOBA
FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES
CÁTEDRA DE PROGRAMACIÓN CONCURRENTE



TRABAJO PRÁCTICO FINAL

Red de Petri Sistema de Procesamiento de Datos

Profesores:

Ventre, Luis Orlando

Ludemann, Mauricio

Carranza, Agustín

INTEGRANTES	DNI	MAIL
Lautaro Callovi	44119027	lautaro.callovi@mi.unc.edu.ar
Alex Joel Rivera	45594556	alex.rivera@mi.unc.edu.ar
Thiago Nahuel Biancucci	45493364	thiago.biancucci@mi.unc.edu.ar
Enzo Laura Surco	44191527	enzo.laura.surco@mi.unc.edu.ar
Luis Mariano Rivera	45432921	luismarianorivera.25@mi.unc.edu.ar
Nicolas Osvaldo Melia	43167517	nicolas.melia@unc.edu.ar



Introducción

El presente trabajo práctico tiene como objetivo modelar y analizar, mediante Redes de Petri y mecanismos de programación concurrente, un sistema de procesamiento de datos. Dicho sistema está compuesto por una cola de arribo de datos, un bus de acceso a un buffer y una unidad de procesamiento, ambos recursos compartidos por todo el sistema, que atiende los datos entrantes mediante tres modos de complejidad alternativos simple, medio y alto cada uno con una cantidad distinta de etapas de procesamiento.

En una primera etapa se realiza el análisis estructural y de comportamiento de la red provista (Figura 1), determinando sus propiedades fundamentales, vivacidad, ausencia de deadlock y seguridad usando la herramienta PIPE, calculando e interpretando sus invariantes de plaza y de transición, vinculando cada uno con el rol de los recursos compartidos y con los tres caminos de procesamiento posibles.

A partir de dicho análisis se diseña e implementa, en lenguaje Java, un monitor que nos permitirá sincronizar el acceso a la concurrencia de forma centralizada, respetando que sea agnóstico a la red que ejecuta, su rol fundamental será la de encargarse de gobernar el disparo de las transiciones sensibilizadas de la Red. Sobre este monitor se incorporan una semántica tiempo débil para las transiciones correspondientes, dos políticas de resolución de conflictos (aleatoria y priorizada) y un esquema de hilos diseñado a partir de los invariantes de transición y los puntos de conflicto y de sincronización de la red.

Finalmente, se presentan los resultados obtenidos a partir de múltiples ejecuciones del sistema, verificando en tiempo de ejecución el cumplimiento de los invariantes de transición usando expresiones regulares analizando el log resultante por ejecución,

Por último, se adjuntan conclusiones del análisis temporal y de distribución de carga entre los modos de procesamiento.

Desarrollo

Se nos proporciona la Red de Petri que modela el sistema de procesamiento de datos a implementar. La misma modela el flujo por el cual se desarrollara cada dato:

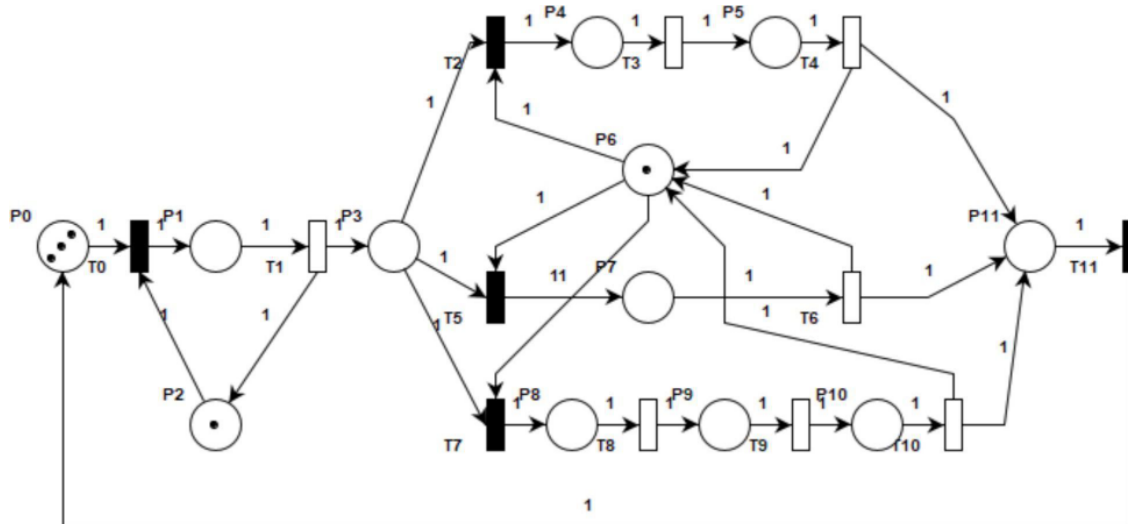


Figura 1 - Red de Petri

Propiedades de la Red

Se realizó el análisis de la Red de Petri usando el software *Pipe v4.3*, y se obtuvieron los siguientes resultados:

Petri net state space analysis results

Bounded	true
Safe	false
Deadlock	false

Figura 2: Propiedades de la Red de Petri

Deadlock

Esta Red de Petri no posee deadlock, lo que significa que nunca va a finalizar por interbloqueo. Esto significa que siempre los datos se procesarán exitosamente.

Vivacidad

La Red de Petri no posee deadlock, por lo que también es viva. Esto significa que en cualquier marca posible siempre existe una transición que podamos disparar generando una secuencia. Esto significa que los datos que ingresan por el buffer siempre se procesarán en algún momento.

Seguridad

Una RDP es segura si dada una marca inicial m_0 , para cada marca alcanzable cada plaza contiene cero o un token. Vemos que esta Red no es segura porque en la plaza P_0 podemos tener más de uno, es decir, podemos tener más de un dato en la cola esperando a ser procesado.

Limitada

Una plaza está limitada por una marca inicial m_0 , si para cada marca alcanzable el número de tokens en dicha plaza es menor o igual a un número finito $k \in \mathbb{N}$. Se observa que la presente RDP es limitada, ya que todas las plazas cumplen con esta condición.

Matriz de Incidencia

La matriz de incidencia expresa algebraicamente la estructura de una Red de Petri y permite determinar cómo modifica cada transición el marcado de sus plazas.

Esta matriz tiene dimensiones $m \times n$, donde m representa la cantidad de plazas y n la cantidad de transiciones. Cada elemento $I[i][j]$ indica la variación neta de tokens que se produce en la plaza P_i cuando se dispara la transición T_j .

Se obtiene mediante:

$$I = I^+ - I^-$$

donde:

- I^+ es la matriz de salida e indica los tokens agregados a cada plaza.
- I^- es la matriz de entrada e indica los tokens retirados de cada plaza.

Por lo tanto, el valor de cada elemento se interpreta de la siguiente manera:

- Si $I[i][j] > 0$, el disparo de T_j agrega tokens a P_i .
- Si $I[i][j] = 0$, el disparo de T_j no produce una variación neta en P_i .
- Si $I[i][j] < 0$, el disparo de T_j consume tokens de P_i .

De esta forma, cada columna describe el efecto completo de una transición sobre el marcado de la red, mientras que cada fila muestra cómo puede variar una plaza ante los distintos disparos.

Combined incidence matrix I												
	T0	T1	T10	T11	T2	T3	T4	T5	T6	T7	T8	T9
P0	-1	0	0	1	0	0	0	0	0	0	0	0
P1	1	-1	0	0	0	0	0	0	0	0	0	0
P10	0	0	-1	0	0	0	0	0	0	0	0	1
P11	0	0	1	-1	0	0	1	0	1	0	0	0
P2	-1	1	0	0	0	0	0	0	0	0	0	0
P3	0	1	0	0	-1	0	0	-1	0	-1	0	0
P4	0	0	0	0	1	-1	0	0	0	0	0	0
P5	0	0	0	0	0	1	-1	0	0	0	0	0
P6	0	0	1	0	-1	0	1	-1	1	-1	0	0
P7	0	0	0	0	0	0	0	1	-1	0	0	0
P8	0	0	0	0	0	0	0	0	0	1	-1	0
P9	0	0	0	0	0	0	0	0	0	0	1	-1

Figura 3: Matriz de incidencia

Invariante de transición

Un invariante de transición es una secuencia de transiciones que al dispararse devuelve la red a su estado inicial o equivalente. En el contexto de nuestra Red de Petri

Invariante de transición - Modo medio:

El invariante de transición T0-T1-T2-T3-T4-T11 representa un ciclo completo de un dato que entra al sistema, cruza el bus, y se procesa por el **modo de complejidad media** tomando y devolviendo la unidad de procesamiento P6 y depositando el dato en P11 que una vez sea disparada por T11 volverá a la cola de arriba como un nuevo dato a procesar.

Invariante de transición - Modo simple:

El invariante de transición T0-T1-T5-T6-T11 representa el mismo ciclo pero elegido el **modo de complejidad simple**.

Invariante de transición - Modo alto:

El invariante de transición T0-T1-T7-T8-T9-T10-T11 representa el ciclo completo elegido el **modo de complejidad alta**.

Vemos que en nuestra Red de Petri tenemos las siguientes T-Invariantes:

T0	T1	T10	T11	T2	T3	T4	T5	T6	T7	T8	T9
1	1	0	1	1	1	1	0	0	0	0	0
1	1	0	1	0	0	0	1	1	0	0	0
1	1	1	1	0	0	0	0	0	1	1	1

Figura 4: Invariantes de transición

Podemos observar 3 invariantes de transición (T - invariantes), los cuales están representados para cada fila de la matriz. Cada uno de ellos describe secuencias cerradas de transiciones, representan diferentes modos de procesamiento de datos en el sistema.

Invariantes de plaza

Un invariante de plaza es una propiedad que garantiza que la suma de los tokens de un cierto conjunto de plazas se mantiene constante independientemente de qué transiciones se activen, es decir, se mantiene constante a lo largo del tiempo.

En nuestra Red de Petri tenemos los siguientes invariantes de plaza:

P0	P1	P10	P11	P2	P3	P4	P5	P6	P7	P8	P9
1	1	1	1	0	1	1	1	0	1	1	1
0	1	0	0	1	0	0	0	0	0	0	0
0	0	1	0	0	0	1	1	1	1	1	1

Figura 5: Invariantes de plaza



Estas invariantes pueden ser representadas por sus respectivas ecuaciones, también obtenidas con el uso de la herramienta Pipe.

$$M(P_0) + M(P_1) + M(P_3) + M(P_4) + M(P_5) + M(P_7) + M(P_8) + M(P_9) + M(P_{10}) + M(P_{11}) = 3$$

$$M(P_1) + M(P_2) = 1$$

$$M(P_4) + M(P_5) + M(P_6) + M(P_7) + M(P_8) + M(P_9) + M(P_{10}) = 1$$

Se analizó también cada P-Invariante con el fin de interpretar su significado en la Red de Petri.

P-invariante 1 – Conservación de datos en el sistema

$$M(P_0) + M(P_1) + M(P_3) + M(P_4) + M(P_5) + M(P_7) + M(P_8) + M(P_9) + M(P_{10}) + M(P_{11}) = 3$$

Este invariante de plaza representa que los datos se conservan dentro del sistema, donde la suma de marcas en las plazas permanece constante e igual a 3, indicando que el sistema no crea ni quita datos durante su ejecución.

P-invariante 2 – Recurso compartido: bus de acceso

$$M(P_1) + M(P_2) = 1$$

Este invariante modela el uso del bus de acceso al buffer como un recurso exclusivo y establece que el bus solo puede ser utilizado por un único dato a la vez.

P-invariante 3 – Unidad de procesamiento

$$M(P_4) + M(P_5) + M(P_6) + M(P_7) + M(P_8) + M(P_9) + M(P_{10}) = 1$$

Este invariante representa la unidad de procesamiento del sistema como un recurso compartido, garantizando que no se puede procesar más de un dato al mismo tiempo.

Tabla de Estados

Los estados del sistema se corresponden con las plazas de la red de Petri, representando las distintas situaciones en las que puede encontrarse un dato o un recurso durante la ejecución del sistema.

Plaza	Estado
P0	Arribo de datos al sistema



P1	Dato accediendo al buffer
P2	Recurso del sistema
P3	Dato almacenado en el buffer de entrada
P4	Primer etapa de la complejidad media
P5	Segunda etapa de la complejidad media
P6	Unidad de procesamiento libre
P7	Única etapa de la complejidad simple
P8	Primer etapa de la complejidad alta
P9	Segunda etapa de la complejidad alta
P10	Tercera etapa de la complejidad alta
P11	Datos saliendo del sistema

Tabla de Eventos

Los eventos del sistema se modelan mediante las transiciones de la red de Petri, representando los cambios de estado que experimentan los datos durante su recorrido por el sistema de procesamiento.

Transición	Evento
T0	Arribo de nuevo dato al sistema
T1	Acceso de un dato al buffer por bus
T2	Procesamiento medio - etapa 1 (inicio)
T3	Procesamiento medio - etapa 2
T4	Procesamiento medio - etapa 3 (final)
T5	Procesamiento simple - etapa 1 (inicio)
T6	Procesamiento simple - etapa 2 (final)
T7	Procesamiento alto - etapa 1 (inicio)
T8	Procesamiento alto - etapa 2
T9	Procesamiento alto - etapa 3



T10	Procesamiento alto - etapa 4 (final)
T11	Salida del dato procesado

Cantidad máxima de hilos en el sistema

En base al paper proporcionado por la cátedra sobre **Algoritmos para determinar cantidad y responsabilidad de hilos en sistemas embebidos modelados con Redes de Petri S3PR** se desarrolló su metodología para esta RDP. Partimos determinando plazas que no son de acción en nuestro sistema:

- . Plaza idle = {P0}
- . Plaza de restricción = {P2}
- . Plaza de recurso = {P6}

1) Los invariantes de transición del sistema son:

$$IT_1 = \{T0, T1, T2, T3, T4, T11\}$$

$$IT_2 = \{T0, T1, T5, T6, T11\}$$

$$IT_3 = \{T0, T1, T7, T8, T9, T10, T11\}$$

2) Los conjunto de plazas PI para cada IT son:

$$PI_1 = \{P0, P1, P2, P3, P4, P5, P6, P11\}$$

$$PI_2 = \{P0, P1, P2, P3, P6, P7, P11\}$$

$$PI_3 = \{P0, P1, P2, P3, P6, P8, P9, P10, P11\}$$

3) Los conjuntos PA (Plazas de Acción) de cada PI son:

$$PA_1 = \{P1, P3, P4, P5, P11\}$$

$$PA_2 = \{P1, P3, P7, P11\}$$

$$PA_3 = \{P1, P3, P8, P9, P10, P11\}$$

Los conjuntos de esta sección se obtienen de los PA quitando las plazas mencionadas al inicio (las cuales no representan una acción en el sistema).

4) Conjunto de estados MA del conjunto de plazas PA, donde PA:

$$PA = \{P1, P3, P4, P5, P7, P8, P9, P10, P11\}$$

Mientras que MA queda conformado por la siguiente tabla:

P1	P3	P4	P5	P10	P7	P8	P9	P11	SUMA
0	0	0	0	0	0	0	0	0	0

1	0	0	0	0	0	0	0	0	1
0	1	0	0	0	0	0	0	0	1
1	1	0	0	0	0	0	0	0	2
0	1	1	0	0	0	0	0	0	2
0	3	0	0	0	0	0	0	0	3
1	1	0	0	0	1	0	0	0	3
0	2	0	0	0	0	0	1	0	3
0	1	0	0	0	0	0	0	1	2
0	0	0	0	0	0	0	0	1	1

Explicación del estado:

- Estado inicial: Los 3 datos están en P0.
- Un dato accede al bus (P1).
- El primer dato pasa al buffer de entrada (P3).
- Un segundo dato entra al bus (P1), el primero espera en P3.
- Dato 1 procesando en etapa media (P4). Dato 2 en buffer.
- MÁXIMO: Los 3 datos entraron rápido y esperan en el buffer.
- MÁXIMO: 1 en bus (P1), 1 en buffer (P3), 1 en CPU simple (P7).
- MÁXIMO: 2 en buffer (P3), 1 en etapa CPU alta (P9).
- 1 dato en buffer (P3), 1 dato saliendo (P11).
- El último dato está por salir del sistema.

De todos los marcados presentados en la tabla, buscamos el máximo. Este valor será el que determine la cantidad máxima de hilos simultáneos en el sistema, siendo en este caso, 3 hilos.

Ahora, aplicando el algoritmo de la sección 4.2 del paper, determinaremos los segmentos de ejecución y responsabilidades del sistema.

Observamos que las 3 invariantes de transición cumplen con la **condición del caso 2**, por ende determinamos los segmentos pertinentes:

- Segmento previo al fork como S_A y $PS_A = \{P1, P3\}$
- Segmento superior como S_B y $PS_B = \{P4, P5\}$
- Segmento medio como S_C y $PS_C = \{P7\}$
- Segmento inferior como S_D y $PS_D = \{P8, P9, P10\}$

Además, se observa que las 3 IT cumplen con la condición del caso 3, por lo cual se define el último segmento como S_E y $PS_E = \{P11\}$.

Determinados estos segmentos, aplicamos los siguientes pasos del algoritmo, el cual implica determinar los marcados máximos de los conjuntos de marcados MS para las plazas de acción de cada segmento de ejecución:



$$Max(MS_A) = 3; Max(MS_B) = 1; Max(MS_C) = 1; Max(MS_D) = 1, Max(MS_E) = 1$$

La suma de todos estos determina la cantidad máxima de hilos en el sistema, que en este caso es de 7.

Según el algoritmo S3PR, el sistema admite un máximo de siete (7) hilos simultáneos. Este valor resulta de sumar los requerimientos de cada segmento de ejecución:

$$Max(MS_A) = 3; Max(MS_B) = 1; Max(MS_C) = 1; Max(MS_D) = 1, Max(MS_E) = 1$$

Este algoritmo responde a la pregunta: **¿cuántos hilos/agentes puede requerir el modelo para cubrir su máximo estructural por segmento?** No responde por sí solo a la pregunta: **¿cuántos hilos/agentes minimizan el tiempo de esta implementación?**

La diferencia es importante, ya que en este sistema de procesamiento los hilos/agentes de entrada no ejecutan una acción computacional independiente: solicitan consecutivamente T0 y T1 al monitor, mientras el bus unitario P2 impide que dos datos atraviesen efectivamente ese tramo al mismo tiempo.

Por eso se evaluaron ambas configuraciones. Los siete hilos representan la aplicación del máximo estructural sugerido por el paper; los cinco hilos representan una configuración reducida que conserva una instancia por cada responsabilidad. La medición experimental permite decidir si el paralelismo potencial del máximo estructural se traduce en una ventaja práctica en esta implementación concreta.

La configuración de cinco hilos asigna una responsabilidad a cada segmento:

Responsabilidad	Secuencia	Hilos
Entrada	T0, T1	1
Procesamiento medio	T2, T3, T4	1
Procesamiento simple	T5, T6	1
Procesamiento alto	T7, T8, T9, T10	1
Salida	T11	1

La configuración de siete hilos no agrega responsabilidades nuevas. Solo utiliza tres hilos en lugar de un hilo para ejecutar la secuencia T0 y T1.

Aunque P0 contiene inicialmente tres marcas, esas marcas representan datos disponibles en la cola de arribo. No significan que tres datos puedan atravesar simultáneamente el segmento de entrada.

El $P_{inv} = M(P1) + M(P2) = 1$. Nos indica que P2, el bus de entrada, es un recurso unitario. T0 consume ese recurso y T1 lo devuelve. Mientras un dato se encuentra en P1, otro hilo no puede disparar un nuevo T0. Por eso el segmento de entrada [T0, T1] admite varios hilos en espera, pero solo un avance efectivo a la vez.

Metodología Experimental

- Configuraciones: 5 y 7 hilos.
- Políticas: aleatoria y priorizada.
- Corridas registradas: 30 por configuración, 120 en total.
- Carga: 200 T-invariantes por corrida.
- Ejecución: una JVM nueva por corrida y configuraciones en orden mezclado.
- Entorno: OpenJDK 21 sobre Linux x86_64.
- Validación: código de salida cero, 200 invariantes completadas y aceptación de RELog.java.

Política	Hilos	Media	Mediana	Desv. estándar	Rendimiento
Aleatoria	5	30.808 s	30.798 s	0.691 s	6,49 inv/s
Aleatoria	7	30.645 s	30.656 s	0.493 s	6,53 inv/s
Priorizada	5	27.000 s	26.893 s	0.619 s	7,41 inv/s
Priorizada	7	26.887 s	26.922 s	0.313 s	7,44 inv/s

Todas las configuraciones completaron 200 invariantes. Con política priorizada, la composición $[IT1, IT2, IT3]$ fue prácticamente idéntica. Con política aleatoria hubo pequeñas variaciones en esa composición: siete hilos promediaron más invariantes simples y menos invariantes medios. Esa variación puede explicar parte de una diferencia temporal de sólo 163 ms y refuerza que no debe atribuirse directamente a la cantidad de hilos.

La cantidad de hilos se definió combinando el análisis estructural de la Red de Petri con una validación experimental. Según el algoritmo del paper, el marcado máximo del segmento de entrada permite asignar tres hilos y conduce a un máximo estructural de siete hilos totales.

Sin embargo, el $P_{inv} = M(P1) + M(P2) = 1$ establece que el bus de entrada es unitario, por lo que solo un dato puede recorrer efectivamente el segmento de entrada $[T0, T1]$ a la vez en esta implementación. En 30 corridas por configuración, materializar el máximo con dos hilos de entrada adicionales redujo el tiempo medio solo un 0,53 % con política aleatoria y un 0,42 % con política priorizada (*en una prueba de benchmark*).

En consecuencia, se seleccionaron cinco hilos como configuración suficiente: un hilo por cada segmento en la RdP. El segmento inicial (S_A , resaltado en rojo) utiliza un (1) hilo para gestionar la adquisición de datos y el buffer (P_1 y P_3). Los cuatro (4) hilos restantes aseguran la exclusión mutua en las etapas de procesamiento (un hilo por cada modo de procesamiento) y un hilo a la salida.

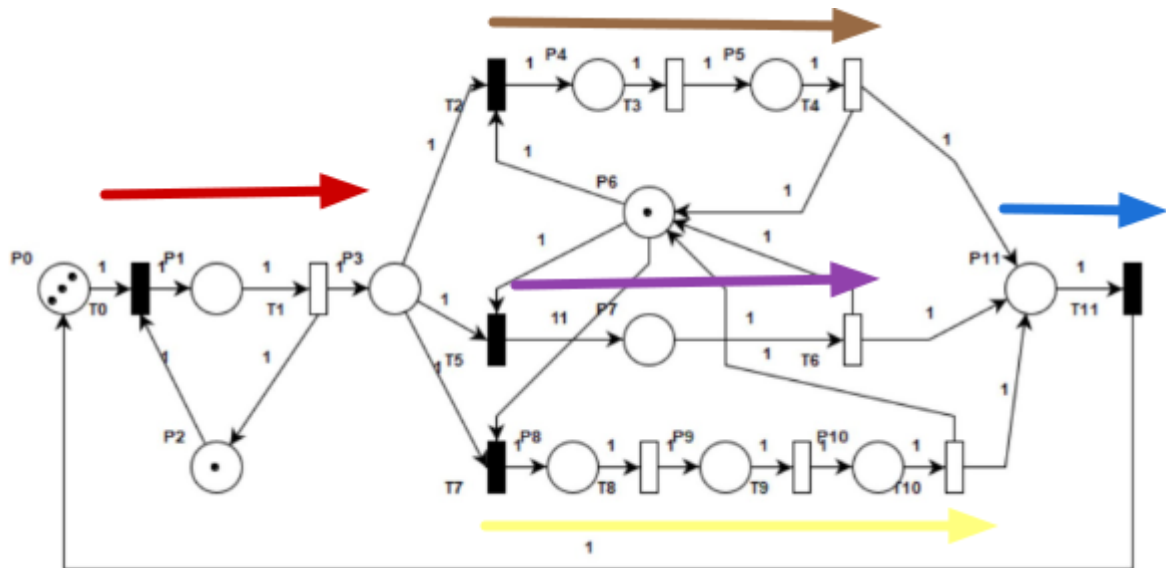


Figura 6: Cantidad de hilos por segmento

Análisis de Tiempos

Con el propósito de evaluar el rendimiento temporal del sistema, se realiza un análisis experimental orientado a medir el tiempo promedio de ejecución bajo diferentes configuraciones y políticas de resolución de conflictos. Asimismo, se verifica que los tiempos registrados cumplan con el rango de ejecución requerido, comprendido entre 20 y 40 segundos.

Modelo Analítico: Pipeline de Tres Etapas

El sistema funciona como un pipeline de tres etapas concurrentes que se solapan en el tiempo gracias al uso de recursos y buffers independientes:

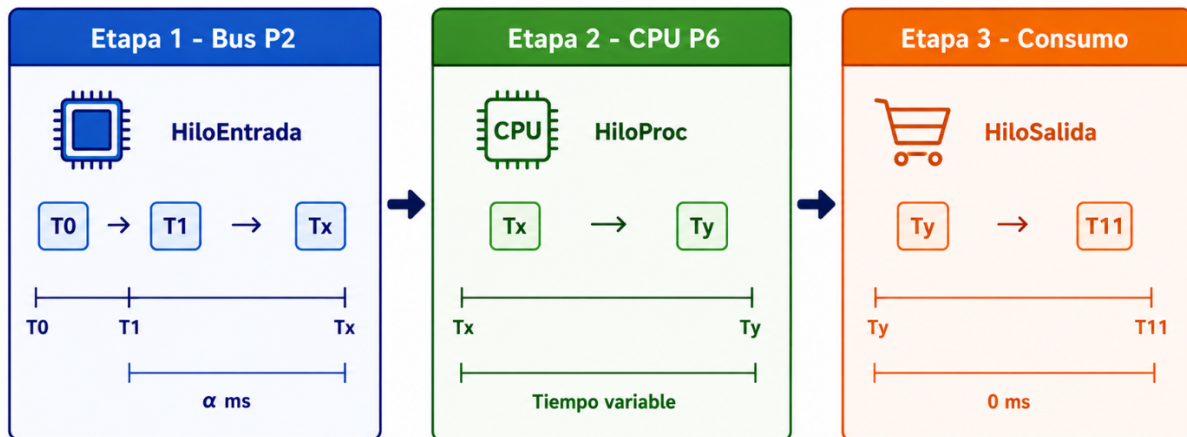


Figura 7: Pipeline de Tres Etapas

Aunque el pipeline consta de tres etapas en la Red, la etapa 3 es instantánea (0 ms) dado que la transición T11 es inmediata. Por lo tanto, no aporta demora en el flujo del sistema, simplificando el análisis de cuello de botella a la relación entre la etapa 1 y la etapa 2.

Costo temporal de cada etapa por dato (en función de EFT), o sea $T_i(\alpha)$:

Etapa 1 - Entrada (Bus P2):

$$T0(0\text{ ms}) + T1(\alpha) = \alpha \times 1\text{ dato}$$

Costo total de entrada: $200 \times \alpha$

Etapa 2 - Procesamiento (CPU P6):

Modo	Transiciones temporales	Costo por dato
Simple	T6	$1 \times \alpha$
Media	T3 + T4	$2 \times \alpha$
Alta	T8 + T9 + T10	$3 \times \alpha$

Etapa 3 - Salida

$T_{11} = 0 \text{ ms}$, al ser una transición instantánea no representa un cuello de botella en nuestra Red.

Fórmula general del tiempo total

Definimos el factor de carga como la cantidad total de etapas temporales que la unidad de procesamiento debe ejecutar secuencialmente:

$$\text{Factor de carga} = N_{\text{simple}} \times 1 + N_{\text{media}} \times 2 + N_{\text{alta}} \times 3$$

Como el costo secuencial de la CPU siempre es mayor o igual al costo de entrada, la CPU es el cuello de botella y el tiempo total es:

$$T_{\text{total}} \approx \alpha \times \text{Factor de carga}$$

Cotas Teóricas Extremas

Cota Inferior (todos los datos por complejidad simple):

$$\begin{aligned} \text{Factor de carga mínimo} &= 200 \times 1 = 200 \\ T_{\text{min}} &= \alpha \times 200 \end{aligned}$$

Esta cota es meramente teórica, en la práctica la concurrencia entre los diferentes hilos de procesamiento y el scheduling de la JVM impiden que un solo modo acapare la totalidad de los datos. Además que la estructura de la Red y su Marcado evita problemas de inanición.

Cota Superior (todos los datos por complejidad alta):

$$\begin{aligned} \text{Factor de carga máximo} &= 200 \times 3 = 600 \\ T_{\text{max}} &= \alpha \times 600 \end{aligned}$$

Igualmente, representa el límite superior teórico si no hubiese distribución aleatoria.

Evidencia Empírica y Validación del Modelo

Para validar el modelo teórico y verificar el comportamiento del sistema ante diferentes ventanas temporales, se recolectaron los datos de 30 ejecuciones (10 ejecuciones por cada valor de $\alpha \in \{50, 75, 100\}$, divididas equitativamente entre las políticas Aleatoria y Prioritaria).

Comparación de Políticas (Tiempos en ms)

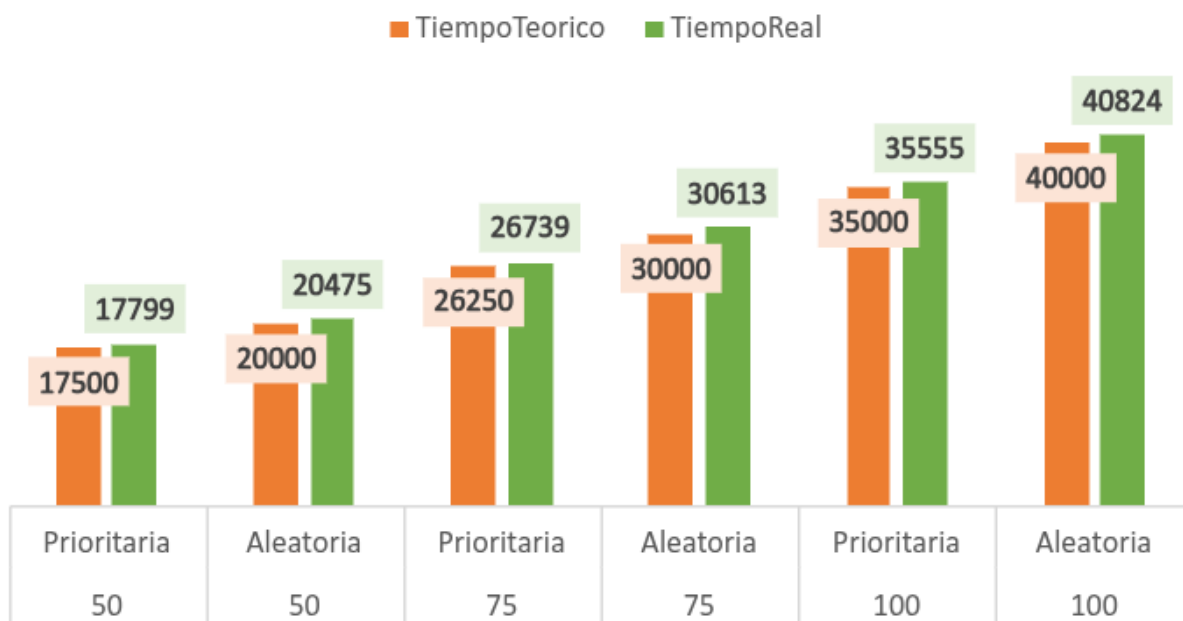


Figura 8: Comparación de tiempos de ejecución en función de α

Tabla Resumen de Resultados Promedio

A continuación se muestran los promedios de tiempo real obtenidos, el tiempo teórico calculado por el modelo y el porcentaje de desviación:

α (ms)	Política	Tiempo Teórico (ms)	Tiempo Real (ms)	Desviación (%)	Cumple (20-40) s
50	Prioritaria	17500	17799	+1.71	NO
50	Aleatoria	20000	20475	+2.38	SI (Límite)
75	Prioritaria	26250	26739	+1.86	SÍ
75	Aleatoria	30000	30613	+2.04	SÍ
100	Prioritaria	35000	35555	+1.59	SÍ
100	Aleatoria	40000	40824	+2.06	NO (Excede)

Análisis de Desviación y Comportamiento del Scheduler

Precisión del Modelo Teórico:

- Con $\alpha = 50$ ms: la desviación promedio es de +1.7% (prioritaria) y +2.4% (aleatoria).
- Con $\alpha = 75$ ms: la desviación promedio es de +1.9% (prioritaria) y +2% (aleatoria).
- Con $\alpha = 100$ ms: la desviación promedio es de +1.6% (prioritaria) y +2% (aleatoria).

Esta desviación representa el overhead de adquisición/liberación de semáforos y la granularidad temporal de `Thread.sleep()`. Gracias a la política de herencia del mutex (Signal-and-Exit), el overhead de sincronización es extremadamente estable y bajo en todas las configuraciones (rondando apenas un ~1.6% a ~2.4%), ya que se evitan re-adquisiciones innecesarias o inanición.



Estabilidad de la Política Prioritaria:

A pesar de priorizar la transición simple (T5), la política prioritaria nunca monopoliza la CPU (la cota inferior teórica de 200 de factor de carga no ocurre). En las 15 ejecuciones documentadas de prioridad, el factor de carga converge de manera muy estable a ~350, lo que representa un balance perfecto entre 100 ciclos simples y 100 ciclos de mayor complejidad (media y alta). Esto se debe a que, mientras un hilo procesa en la CPU (T6, T3/T4 o T8/T9/T10), los datos continúan acumulándose en el buffer P3. Al quedar libre el procesador, si el HiloSimple aún se encuentra durmiendo en su etapa temporal T6, el monitor despacha a los otros dos hilos en espera (Media o Alta), garantizando la no inanición y manteniendo un factor de carga balanceado de procesamiento.

Conclusiones sobre la Restricción Temporal

$\alpha = 50$ ms es INSUFICIENTE: La política prioritaria completa la simulación en un promedio de 17.8 s, incumpliendo el mínimo de 20 s.

$\alpha = 100$ ms es EXCESIVO: La política aleatoria excede el límite superior de 40 s, promediando 40.8 s y alcanzando picos de 41.9 s.

$\alpha = 75$ ms representa una elección óptima y segura:

- Con política prioritaria, el tiempo promedio es de 26.7 s, lo cual cumple perfectamente la restricción.
- Con política aleatoria, el tiempo promedio es de 30.6 s, lo cual también cumple perfectamente la restricción.

Regex

En un sistema concurrente modelado por una Red de Petri, los T-invariantes (representan secuencias de disparo que, al completarse, devuelven el sistema a su marcado original. Verificar que la traza de ejecución respeta estos invariantes es fundamental para demostrar el correcto funcionamiento del monitor de concurrencia: si todos los disparos registrados pueden agruparse en ciclos de T-invariantes completos, se prueba que no hubo disparos incompletos, pérdida de tokens ni violación de la estructura de la red.

El desafío es que, en un entorno multi-hilo, los disparos de distintos hilos se intercalan (interleaving) de forma no determinista. El log no contiene secuencias limpias como T0 T1 T2 T3 T4 T11, sino trazas entrelazadas.

En ese sentido se implementa un algoritmo de reducción recursiva por expresiones regulares que resuelve este problema de forma elegante y formal.

Expresión Regular

(T0)(.*?)(T1)(?!d)(.*?)((T2)(.*?)(T3)(.*?)(T4)|(T5)(.*?)(T6)|(T7)(.*?)(T8)(.*?)(T9)(.*?)(T10))(.?)(T11)(.*?)

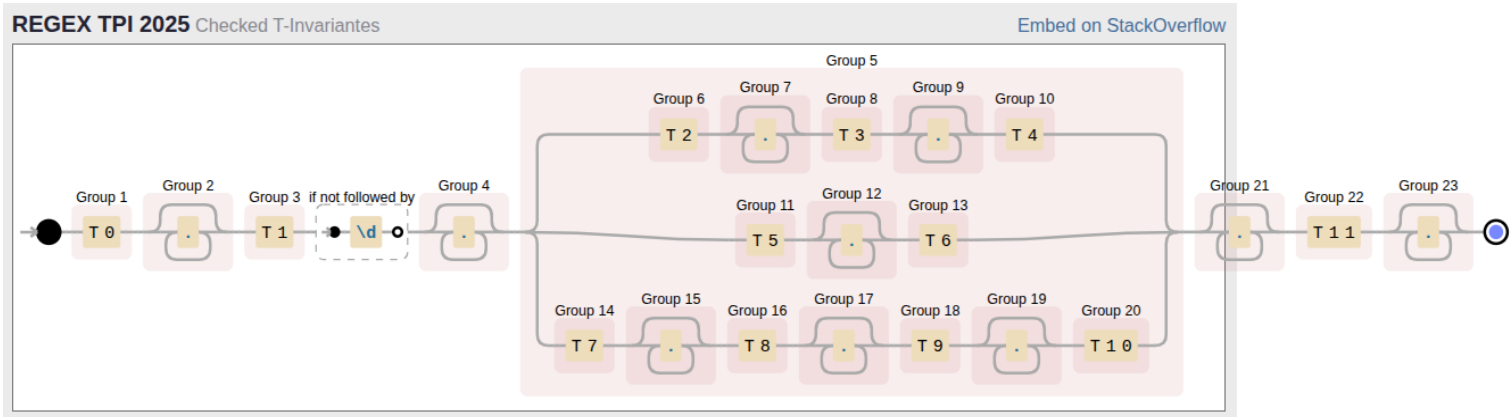


Figura 9: Expresión Regular implementada en [Debuggex](#)

Bloque de Entrada Común

(T0)(.*?)(T1)(?!d)(.*?)

Fragmento	Grupo	Función
(T0)	Grupo 1	Captura literal de la transición de entrada al sistema
(.*?)	Grupo 2	Comodín lazy: captura cualquier cantidad de transiciones intermedias de otros hilos (interleaving) entre T0 y T1, usando la menor cantidad posible de caracteres
(T1)	Grupo 3	Captura literal de la transición de acceso al buffer
(?!d)	—	Lookahead negativo: asegura que después de T1 no haya un dígito, evitando confundir T1 con el prefijo de T10 o T11



(.*?)	Grupo 4	Captura interleaving entre T1 y el inicio del bloque de procesamiento
-------	---------	---

Bloque de Procesamiento

((T2)(.*?)(T3)(.*?)(T4)|(T5)(.*?)(T6)|(T7)(.*?)(T8)(.*?)(T9)(.*?)(T10))

Este es el corazón de la expresión. Modela los tres caminos posibles como alternativas dentro de un grupo contenedor (grupo 5):

Alternativa 1 - Complejidad Media:

(T2)(.*?)(T3)(.*?)(T4)

Fragmento	Grupo	Función
(T2)	Grupo 6	Primera etapa del procesamiento medio
(.*?)	Grupo 7	Interleaving entre T2 y T3
(T3)	Grupo 8	Segunda etapa del procesamiento medio
(.*?)	Grupo 9	Interleaving entre T3 y T4
(T4)	Grupo 10	Finalización del procesamiento medio

Alternativa 2 - Complejidad Simple:

(T5)(.*?)(T6)

Fragmento	Grupo	Función
(T5)	Grupo 11	Inicio del procesamiento simple
(.*?)	Grupo 12	Interleaving entre T5 y T6
(T6)	Grupo 13	Finalización del procesamiento simple

Alternativa 3 - Complejidad Alta:

(T7)(.*?)(T8)(.*?)(T9)(.*?)(T10)



Fragmento	Grupo	Función
(T7)	Grupo 14	Primera etapa del procesamiento alto
(.*?)	Grupo 15	Interleaving entre T7 y T8
(T8)	Grupo 16	Segunda etapa del procesamiento alto
(.*?)	Grupo 17	Interleaving entre T8 y T9
(T9)	Grupo 18	Tercera etapa del procesamiento alto
(.*?)	Grupo 19	Interleaving entre T9 y T10
(T10)	Grupo 20	Finalización del procesamiento alto

Bloque de Salida Común

(.*?)(T11)(.*?)

Fragmento	Grupo	Función
(.*?)	Grupo 21	Interleaving entre el fin del procesamiento y la salida
(T11)	Grupo 22	Captura literal de la transición de salida del dato
(.*?)	Grupo 23	Interleaving residual después del invariante completo (cola)

Algoritmo de Reducción Recursiva

El algoritmo opera bajo el principio de eliminación progresiva. En cada pasada (de izq a der):

1. Se recorre la cadena de la traza buscando todas las coincidencias no solapadas del patrón.
2. Cada coincidencia representa un T-invariante completo encontrado dentro del interleaving.
3. Se remueve el invariante de la cadena, pero se conserva todo el contenido capturado por los grupos comodín (.*?), que corresponden a transiciones de otros hilos intercalados.
4. Se repite el proceso hasta que no se encuentren más coincidencias.

Este algoritmo funciona así, porque en una sola pasada, la regex encuentra coincidencias no solapadas de izquierda a derecha. Esto implica que ciertos invariantes cuyos componentes estaban "atrapados" dentro del interleaving de otros invariantes no pueden ser detectados hasta que los invariantes externos sean removidos primero.

De esta forma se garantiza que todos los invariantes sean eventualmente encontrados, independientemente de cuán profundamente estén entrelazados.

```
anpanman17@HP-Zorin18:~/Escritorio/ProgConcu/HilandoJavaFinal$ java -cp target/classes tpfinal.utils.RELog priority
=====
Verificador RELog - Validación de T-Invariantes
Archivo: log_priority.txt
=====
Log cargado correctamente. Transiciones: 1108
--- Invariantes de transición detectados (reducción recursiva) ---
IT1 - Complejidad media (T0-T1-T2-T3-T4-T11) : 34
IT2 - Complejidad simple (T0-T1-T5-T6-T11) : 129
IT3 - Complejidad alta (T0-T1-T7-T8-T9-T10-T11): 37
Coincidencias (ciclos válidos) : 200
--- Resultado ---
VALIDACION DE INVARIANTES EXITOSA
```

Figura 9: Ejecución de la Regex con política prioritaria

```
anpanman17@HP-Zorin18:~/Escritorio/ProgConcu/HilandoJavaFinal$ java -cp target/classes tpfinal.utils.RELog priority
=====
Verificador RELog - Validación de T-Invariantes
Archivo: log_priority.txt
=====
Log cargado correctamente. Transiciones: 1106
--- Invariantes de transición detectados (reducción recursiva) ---
IT1 - Complejidad media (T0-T1-T2-T3-T4-T11) : 33
IT2 - Complejidad simple (T0-T1-T5-T6-T11) : 128
IT3 - Complejidad alta (T0-T1-T7-T8-T9-T10-T11): 37
Coincidencias (ciclos válidos) : 198
--- Resultado ---
VALIDACION DE INVARIANTES FALLIDA
Transiciones restantes: T0T1T0T1T5T6T2T3T4
```

Figura 10: Ejecución de la Regex con política prioritaria borrando T11 dos veces

En efecto, la traza se reduce a cadena vacía si y sólo si todos los disparos registrados pertenecen a ciclos completos de T-invariantes. Cualquier transición restante nos constituye como evidencia directa de un defecto en el sistema concurrente.



Anexo

Datos de Ejecución

A continuación, se presentan los datos obtenidos de las 30 ejecuciones realizadas (10 ejecuciones para cada configuración de α).

1. Configuración: $\alpha = 50 \text{ ms}$

Política Prioritaria

Ejecución	T_real (ms)	N_simple	N_media	N_alta	Factor	T_teórico (ms)	Desviación
1	17.596	100	58	42	342	17.100	+2.90%
2	17.971	101	48	51	350	17.500	2.69%
3	17.820	100	53	47	347	17.350	+2.71%
4	18.041	100	49	51	351	17.550	+2.80%
5	16.558	100	59	41	341	17.050	+3.03%

Política Aleatoria

Ejecución	T_real (ms)	N_simple	N_media	N_alta	Factor	T_teórico (ms)	Desviación
1	20.298	65	75	60	395	19.750	+2.78%
2	20.732	62	72	66	404	20.200	+2.63%
3	20.296	67	71	62	395	19.750	+2.77%
4	20.567	69	62	69	400	20.000	+2.84%
5	20.480	69	63	68	399	19.950	+2.66%

2. Configuración: $\alpha = 75 \text{ ms}$

Política Prioritaria

Ejecución	T_real (ms)	N_simple	N_media	N_alta	Factor	T_teórico (ms)	Desviación
1	27.028	100	46	54	354	26.650	+1.8%
2	26.839	100	49	51	351	26.325	+1.95%



3	26.261	100	56	44	344	25.800	+1.79%
4	26.563	100	53	47	347	26.025	+2.07%
5	27.003	100	47	53	353	26.475	+1.99%

Política Aleatoria

Ejecución	T_real (ms)	N_simple	N_media	N_alta	Factor	T_teórico (ms)	Desviación
1	30.585	65	69	66	401	30.075	+1.70%
2	30.587	61	77	62	401	30.075	+1.70%
3	30.971	64	67	69	405	30.375	+1.96%
4	30.197	70	65	65	395	29.625	+1.93%
5	30.726	64	70	66	402	30.150	+1.91%

3. Configuración: $\alpha = 100\text{ ms}$

Política Prioritaria

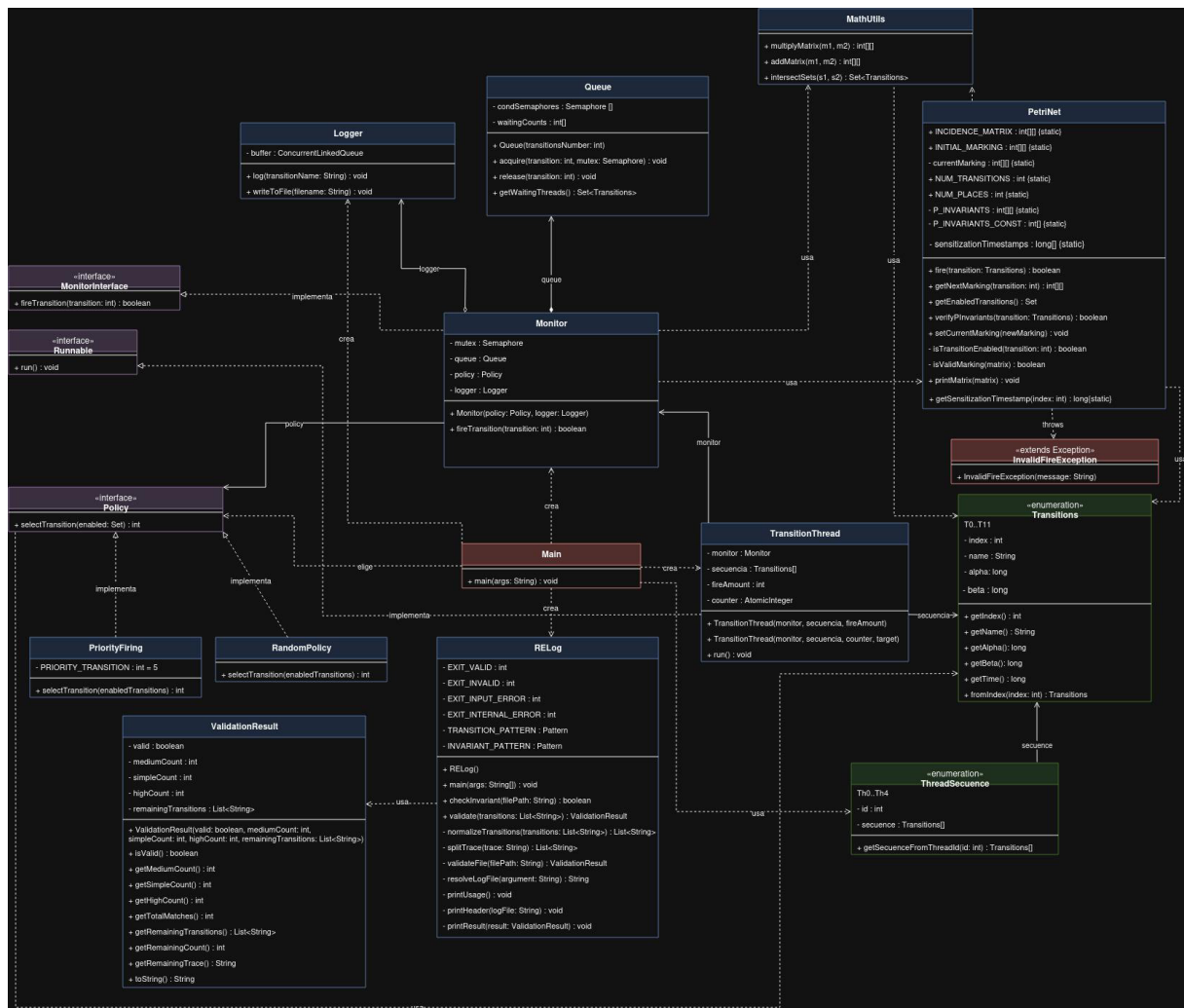
Ejecución	T_real (ms)	N_simple	N_media	N_alta	Factor	T_teórico (ms)	Desviación
1	35.965	100	46	54	354	35.400	+1.6%
2	35.710	100	48	52	352	35.200	+1.45%
3	35.034	100	55	45	345	34.500	+1.55%
4	36.544	100	40	60	360	36.000	+1.51%
5	34.522	103	54	43	340	34.000	+1.54%



Política Aleatoria

Ejecución	T_real (ms)	N_simple	N_media	N_alta	Factor	T_teórico (ms)	Desviación
1	39.883	67	73	60	393	39.300	+1.48%
2	41.937	60	67	73	413	41.300	+1.54%
3	41.945	59	69	72	413	41.300	+1.56%
4	40.109	71	63	66	395	39.500	+1.54%
5	40.246	70	64	66	396	39.600	+1.63%

Diagrama de clases





unc



FCEFN

FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES

Bibliografía y herramientas:

Pipe v4.3.

Paper - Cantidad máxima de hilos en el sistema:

[Algoritmos para determinar cantidad y responsabilidad de hilos en sistemas embebidos modelados con Redes de Petri S 3 PR - Micolini, Ventre](#)

Clases y material proporcionados por la cátedra